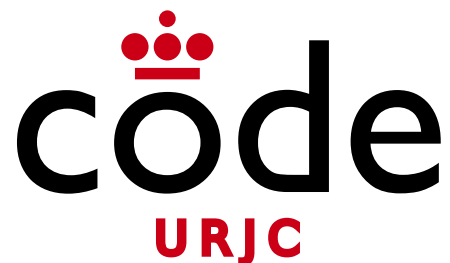


Fundamentos de la Web

Bloque III: Tecnologías de servidor web

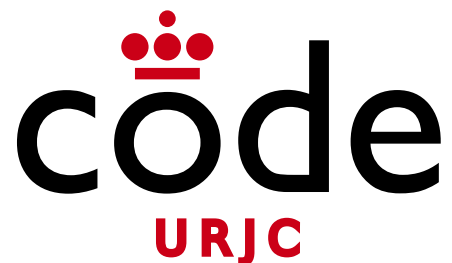
Tema 3.4: Aplicaciones Web con Node



Fundamentos de la Web

Bloque III: Tecnologías de servidor web

Tema 3.4.1: Aplicaciones Web con Express



©2024

Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto, Iván Chicano

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional”
de Creative Commons Disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Aplicaciones web con Node

- Una de las librerías más usadas para el desarrollo de aplicaciones web con Node.js es express

express

<https://expressjs.com>

Aplicación web mínima

ejem1-helloworld

app.js

```
import express from 'express';

const app = express();

app.get('/', (req, res) => {
  res.send('<html><body><h1>Hello World!</h1></body></html>');
});

app.listen(3000, () => console.log('Web ready in http://localhost:3000/'));
```

package.json

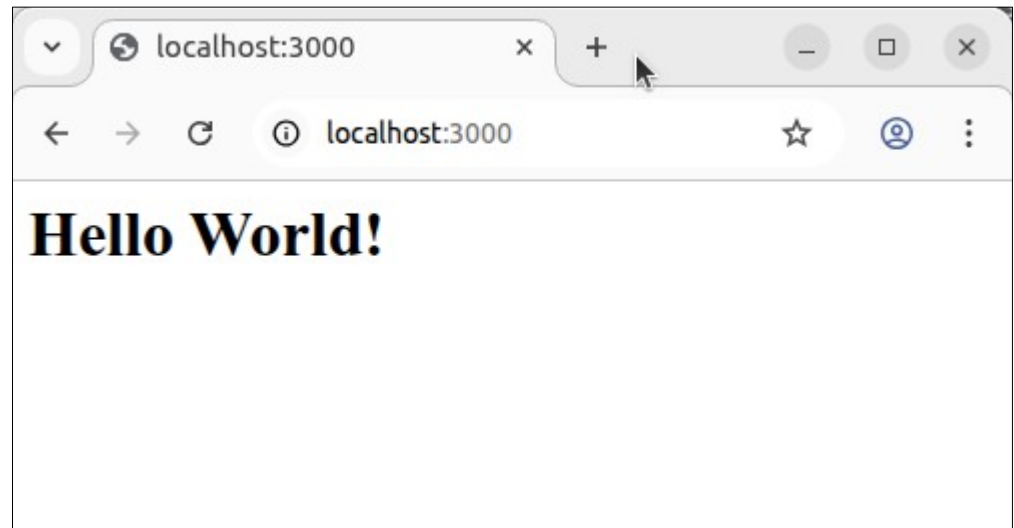
```
{
  "name": "web-ejem1",
  "version": "1.0.0",
  "type": "module",
  "dependencies": {
    "express": "5.1.10"
  }
}
```

Aplicación web mínima

ejem1-helloworld

```
$ npm install
```

```
$ node app.js
```



<http://localhost:3000>

Aplicación web típica

- Comando para facilitar el desarrollo
- Si usamos el comando node para ejecutar la web, por cada cambio de código es necesario **parar y ejecutar** de nuevo la aplicación
- La herramienta **nodemon** detecta los cambios y reinicia la aplicación de forma automática

```
$ sudo npm install -g nodemon
```

```
$ nodemon -e js,html,css app.js
```

Aplicación web típica

- Comando para facilitar el desarrollo
 - Para no tener que escribir el comando cada vez, se define en la sección **scripts** del **package.json**

package.json

```
{
  "name": "web-ejem2",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node src/app.js",
    "watch": "nodemon -e js,html,css src/app.js"
  },
  "dependencies": {
    "express": "5.1.0",
    "mustache-express": "1.3.2"
  }
}
```

\$ npm start

\$ npm run watch

Aplicación web típica

- El código se mueve a la carpeta **/src**
- Se **separan** responsabilidades en el código
 - **app.js**: Configuración de express
 - **router.js**: Funcionalidad de la aplicación
- Se usan **plantillas para generar HTML** (*templates*) (en vez de usar Strings en JavaScript)
- Se configura una carpeta **/public** para servir los ficheros estáticos (imágenes, CSS, html estático...)

Aplicación web típica

- Express es una librería **muy configurable**
- Es necesario configurar de forma **explícita**:
 - Carpeta para servir ficheros estáticos por http
 - Procesado del cuerpo (body) en las peticiones
 - Tecnología de plantillas para HTML (*templates*)

Aplicación web típica

- **Mustache**

- Existen muchas tecnologías para crear páginas web dinámicas (plantillas o *templates*)
- Nosotros usaremos Mustache

```
$ npm install --save mustache-express
```

<https://mustache.github.io/mustache.5.html>

<http://expressjs.com/en/guide/using-template-engines.html>

Aplicación web típica

Express y Mustache

package.json

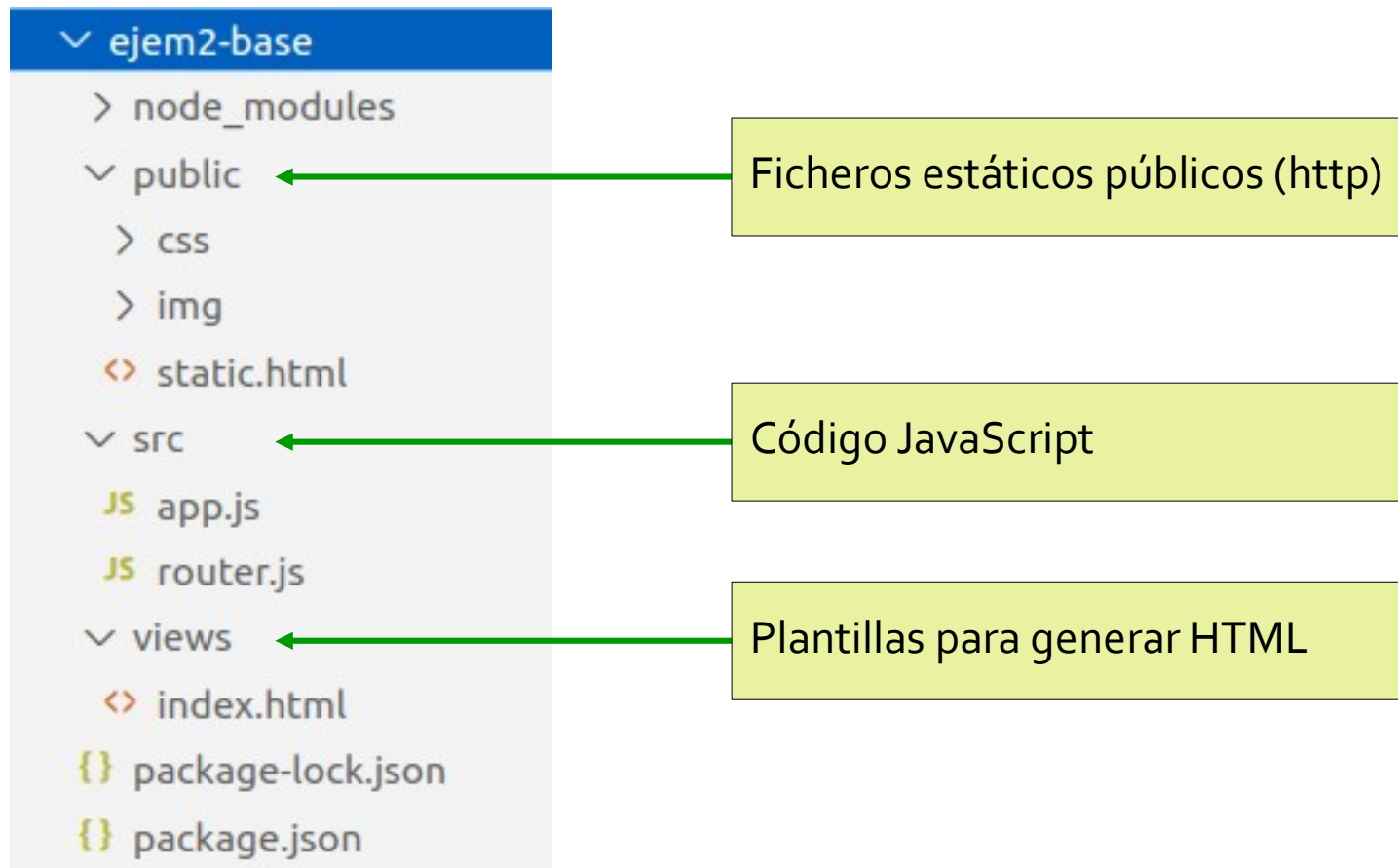
```
{
  "name": "web-ejem2",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node src/app.js",
    "watch": "nodemon -e js,html,css src/app.js"
  },
  "dependencies": {
    "express": "5.1.0",
    "mustache-express": "1.3.2"
  }
}
```

Express y mustache



Aplicación web típica

Estructura de carpetas



Aplicación web típica

app.js

```
import express from 'express';
import mustacheExpress from 'mustache-express';
import bodyParser from 'body-parser';

import router from './router.js';

const app = express();

app.use(express.static('./public'));

app.set('view engine', 'html');
app.engine('html', mustacheExpress(), ".html");
app.set('views', './views');

app.use(bodyParser.urlencoded({ extended: true }));

app.use('/', router);

app.listen(3000, () => console.log('Web ready in http://localhost:3000/'));
```

Funcionalidad de la aplicación

Carpeta pública

Sistema de plantillas

Procesado del body

Aplicación web típica

/src/app.js

```
import express from 'express';

const router = express.Router();
export default router;

router.get('/', (req, res) => {

  res.render('index', {
    name: "World"
  });

});
```

/views/index.html

```
<html>

<head>
<link type="text/css" rel="stylesheet"
  href="/css/styles.css">
</head>

<body>

  </img>

  <h1 class="title">Hello {{name}}</h1>

  <a href="/static.html">Go to static page</a>

</body>

</html>
```

Aplicación web típica

/src/router.js

```
import express from 'express';

const router = express.Router();
export default router;

router.get('/', (req, res) => {

  res.render('index', {
    name: "World"
  });

});
```

/views/index.html

```
<html>

<head>
<link type="text/css" rel="stylesheet"
  href="/css/styles.css">
</head>

<body>

  </img>

  <h1 class="title">Hello {{name}}</h1>

  <a href="/static.html">Go to static page</a>

</body>

</html>
```


Aplicación web típica

/views/index.html

```
<html>

<head>
<link type="text/css" rel="stylesheet"
  href="/css/styles.css">
</head>

<body>

  </img>

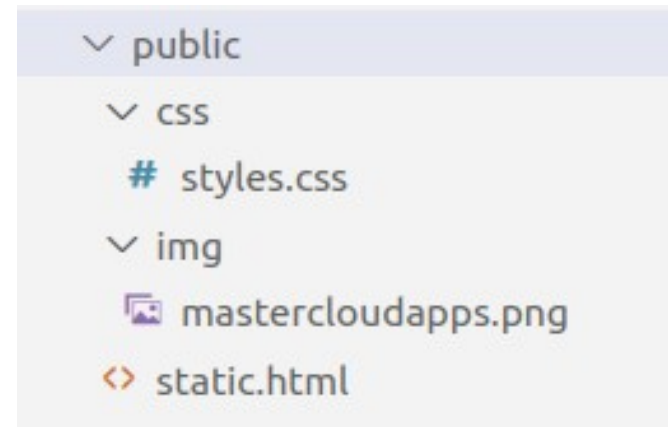
  <h1 class="title">Hello {{name}}</h1>

  <a href="/static.html">Go to static page</a>

</body>

</html>
```

El contenido de la carpeta **public** se referencia desde las plantillas con ruta absoluta (empezando por /)



Generación de HTML con Mustache

- **Funcionalidades básicas**
 - Generación condicional de HTML si una expresión es true
 - Generación de listas HTML con el contenido de elementos tipo lista
 - Generación de tablas HTML con el contenido de un objeto
 - Uso de cabecera y pie comunes a varias páginas

Generación de HTML con Mustache

ejem3-mustache

src/router.js

```
router.get('/conditional', (req, res) => {
  res.render('conditional', {
    name: "World",
    hello: true
  });
});
```

views/conditional.html

```
<html>
  <body>

    {{#hello}}
      <p>Hello,</p>
    {{/hello}}

    {{^hello}}
      <p>Goodbye,</p>
    {{/hello}}

    <p>{{name}}</p>
  </body>
</html>
```

Generación condicional

Si **hello=true** se mostrará Hello.

Si **hello=false** se mostrará Goodbye.

También se puede usar para detectar elementos ciertos o falsos (lista vacía, o, undefined, null...)

```
<html>
  <body>
    <p>Hello,</p>
    <p>World</p>
  </body>
</html>
```

Generación de HTML con Mustache

ejem3-mustache

src/router.js

```
router.get('/list', (req, res) => {
  res.render('list', {
    colors: ["red", "green", "blue"]
  });
});
```

views/list.html

```
<html>
  <body>
    <p>Colors in list:</p>
    <ul>
      {{#colors}}
        <li>{{.}}</li>
      {{/colors}}
    </ul>
  </body>
</html>
```

Repite la etiqueta por cada
element de la lista

El punto hace referencia al objeto
en esa iteración

```
<html>
  <body>
    <p>Colors in list:</p>
    <ul>
      <li>red</li>
      <li>green</li>
      <li>blue</li>
    </ul>
  </body>
</html>
```

Generación de HTML con Mustache

ejem3-mustache

src/router.js

```
router.get('/object', (req, res) => {
  res.render('object', {
    name: {
      first: "Michael",
      last: "Jackson"
    }
  });
});
```

Puedes usar la notación punto para acceder a las propiedades de un objeto por su clave

views/object.html

```
<html>
  <body>
    <p>{{name.first}} {{name.last}}</p>
  </body>
</html>
```



```
<html>
  <body>
    <p>Michael Jackson</p>
  </body>
</html>
```

Generación de HTML con Mustache

ejem3-mustache

src/router.js

```
router.get('/beatles', (req, res) => {
  res.render('beatles', {
    beatles: [
      { firstName: "John", lastName: "Lennon" },
      { firstName: "Paul", lastName: "McCartney" },
      { firstName: "George", lastName: "Harrison" },
      { firstName: "Ringo", lastName: "Starr" }
    ], ...
  })
})
```

views/beatles.html

```
<html>
  <body>
    <table>
      {{#beatles}}
        <tr>
          <td>{{firstName}}</td>
          <td>{{lastName}}</td>
        </tr>
      {{/beatles}}
    </table>
```

...

Si se usa un array con objetos dentro que comparten las mismas propiedades, puede iterarse sobre los objetos usando los nombres de sus propiedades

Generación de HTML con Mustache

ejem3-mustache

```
<html>
  <body>
    <table>
      <tr>
        <td>John</td>
        <td>Lennon</td>
      </tr>
      <tr>
        <td>Paul</td>
        <td>McCartney</td>
      </tr>
      <tr>
        <td>George</td>
        <td>Harrison</td>
      </tr>
      <tr>
        <td>Ringo</td>
        <td>Starr</td>
      </tr>
    </table>
```

...

Generación de HTML con Mustache

ejem3-mustache

src/router.js

```
router.get('/beatles', (req, res) => {
  res.render('beatles', {
    beatles: [
      { firstName: "John", lastName: "Lennon" },
      { firstName: "Paul", lastName: "McCartney" },
      { firstName: "George", lastName: "Harrison" },
      { firstName: "Ringo", lastName: "Starr" }
    ],
    name: function () {
      return this.firstName + " " + this.lastName;
    }
  });
});
```

Si se implementa una **función**, se le llamará en cada iteración

views/beatles.html

```
...
<ul>
  {{#beatles}}
    <li>{{name}}</li>
  {{/beatles}}
</ul>
...
```


Generación de HTML con Mustache

```
...  
<ul>  
  <li>John Lennon</li>  
  <li>Paul McCartney</li>  
  <li>George Harrison</li>  
  <li>Ringo Starr</li>  
</ul>  
...
```

Generación de HTML con Mustache

ejem3-mustache

- Cabecera y pie común en todas las plantillas
 - Las plantillas pueden incluir el contenido de otras plantillas con `{{>plantilla}}`

views/page.html

```
{{>header}}
<p>Main content</p>
{{>footer}}
```

views/header.html

```
<html>
  <body>
    <h1>Welcome {{username}}</h1>
```

views/footer.html

```
    <p>Footer</p>
  </body>
</html>
```

Generación de HTML con Mustache

ejem3-mustache

- Cabecera y pie común en todas las plantillas
 - Podemos crear una función *middleware* para añadir datos al modelo en todas las peticiones
 - Funciones que permiten **ejecutar código antes de mandar la respuesta** a una petición
 - También permite modificar la petición y la respuesta

Generación de HTML con Mustache

ejem3-mustache

- Cabecera y pie común en todas las plantillas
 - Exportamos una función default
 - Metemos *username* en el objeto **locals** dentro de la respuesta (*res*)

src/defaultModel.js

```
export function defaultModel(req, res, next) {
  res.locals.username = "Juan";
  next();
}
```

Generación de HTML con Mustache

ejem3-mustache

- Cabecera y pie común en todas las plantillas
 - Usamos el nuevo middleware en nuestra configuración de `app.js`

src/app.js

```
...
import { defaultModel } from './defaultModel.js';
...
const app = express();
...
app.use(defaultModel);
...
```

- Ahora todos los templates para todas las peticiones podrán usar `{{username}}`

Proceso de formularios y enlaces

- **Formas de enviar información del navegador al servidor**
 - **Mediante formularios HTML:** La información la introduce manualmente el usuario
 - **Insertando información en la URL de enlaces:** La información la incluye el desarrollador para que esté disponible cuando el usuario pulse el enlace

Proceso de formularios y enlaces

- Existen dos formas de enviar la información de un formulario al servidor

- Método GET (Por defecto)

```
<form action="url_router">
...
</form>
```

- Método POST

```
<form method="post" action="url_router">
...
</form>
```

- action:** URL del router que será ejecutado al enviar los datos al servidor pulsando el botón de enviar (*submit*)

Proceso de formularios y enlaces

- Creación de formularios en HTML
 - Tiene que existir al menos un **botón para enviar** los datos del formulario

```
<form action="url_router">
  ...
  <input type="submit" value="Enviar">
</form>
```

Botón con texto

```
<form action="url_router">
  ...
  <input type="image" src="imagen.png">
</form>
```

Botón gráfico

Proceso de formularios y enlaces

- Existen dos formas de enviar la información de un formulario al servidor
 - **Método GET (Por defecto)**
 - Se usa el método GET del protocolo HTTP
 - El navegador incluye la información del formulario en la URL que solicita al servidor
 - **Método POST**
 - Se usa el método POST del protocolo HTTP
 - El navegador incluye la información del formulario en el cuerpo de la petición (no es visible por el usuario)

Proceso de formularios y enlaces

- Procesar diferentes métodos HTTP en express

```
app.METHOD(PATH, HANDLER)
```

- **app** es una instancia de express. También puede ser un router
- **METHOD** es el método de solicitud HTTP
- **PATH** es la ruta de acceso en el servidor
- **HANDLER** es la función que se ejecuta cuando se accede a la ruta

Proceso de formularios y enlaces

- Acceso a los datos en el servidor
 - Cuando el formulario es GET los valores se obtienen de `req.query`

public/index.html

```
<form action="greeting">
  <p>Hello to:</p>
  <input type="text" name="userName"/>
  <input type="text" name="country"/>
  <input type="submit" value="Send"/>
</form>
```

Parámetros con el valor
del campo de texto del
formulario

```
router.get('/greeting', (req, res) => {

  res.render('greeting', {
    name: req.query.userName,
    country: req.query.country
  });
});
```

Proceso de formularios y enlaces

- Acceso a los datos en el servidor
 - Cuando el formulario es POST los valores se obtienen de **req.body**

public/index.html

```
<form action="login" method="post">
  <p>Login:</p>
  <input type="text" name="userName"/>
  <input type="password" name="password"/>
  <input type="submit" value="Send"/>
</form>
```

src/app.js

```
router.post('/login', (req, res) => {

  let password = "1234";

  let success = req.body.password === password;

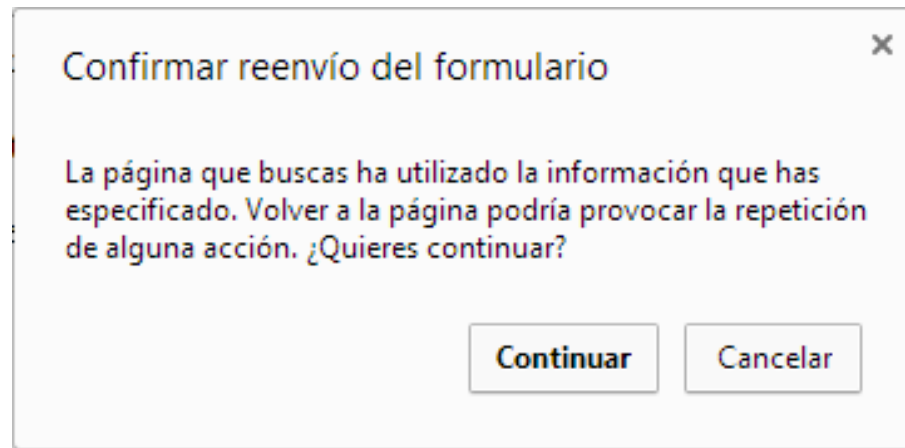
  res.render('login', {
    success,
    name: req.body.userName,
  });
});
```

Proceso de formularios y enlaces

- Se recomienda usar el método POST en formularios
 - Las URLs quedan más limpias (porque no incluyen los parámetros)
 - La página que aparece al enviar el formulario se puede añadir a la lista de marcadores del navegador

Proceso de formularios y enlaces

- Se recomienda usar el método POST en formularios
 - Si se intenta recargar la página del formulario el navegador muestra un aviso al usuario cargar la página tenga efectos no deseados.



Ejercicio 1

- Crear una página **html estática** que muestre un **formulario** para enviar al servidor información de nuevos anuncios.
- En el formulario debe aparecer:
 - Campo de texto para el **nombre** del usuario (*user*)
 - Campo de texto para el **asunto** del mensaje (*title*)
 - Área de texto para el **cuerpo** del mensaje (*text*)
 - **Botón de envío**
- Implementar un método en un fichero **router** que sea ejecutado al pulsar el botón de envío del formulario y recoja los datos del formulario.
- Diseñar una **plantilla** que muestre el anuncio que se ha enviado al servidor.

Ejercicio 1

/public/index.html

Nombre:

Asunto:

Comentario:

router.js

show_post.html

Nombre: Pedro
Asunto: Vendo moto
Comentario: Vendo moto Honda roja del 95

Proceso de formularios y enlaces

- **Formas de enviar información del navegador al servidor**
 - **Mediante formularios HTML:** La información la introduce manualmente el usuario
 - **Insertando información en la URL de enlaces:** La información la incluye el desarrollador para que esté disponible cuando el usuario pulse el enlace

Proceso de formularios y enlaces

- Insertando información en la URL de los enlaces

- Es habitual que los desarrolladores incluyan información en la URL para que esté disponible en el servidor cuando el usuario pulsa el enlace
- Formato:

URL con parámetros

```
http://www.traumainforma.com/nuevoUsuario?option=com_weblinks&view=category&lang=es
```

- Los parámetros se incluyen al final de la URL separados con ? (query)
- Los parámetros se separan entre sí con &
- Cada parámetro se codifica como **nombre=valor**

Proceso de formularios y enlaces

- **Insertando información en la URL de los enlaces**
 - Formato: Codificación de los nombres y los valores
 - Los caracteres alfanuméricos "a" hasta "z", "A" hasta "Z" y "0" hasta "9" se quedan igual
 - Los caracteres especiales ".", "-", "*", y "_" se quedan igual
 - El carácter espacio " " es convertido al signo "+"
 - Todos los otros caracteres son codificados en uno o más bytes. Después cada byte es representado por la cadena de 3 caracteres "%xy", donde xy es la representación en hexadecimal del byte

Proceso de formularios y enlaces

- Insertando información en la URL de los enlaces

Parámetros:

Nombre: direccion

Valor: C\ Pepe, nº 3

Nombre: poblacion

Valor: Madrid

URL con parámetros:

<http://www.micasa.com/nueva?direccion=C%5C+Pepe%2C+n%BA+3&poblacion=Madrid>

Proceso de formularios y enlaces

ejem5-link-urlparam

- Insertando información en la URL de los enlaces
 - Para acceder a la información se usa el mismo mecanismo que para leer los campos del formulario GET

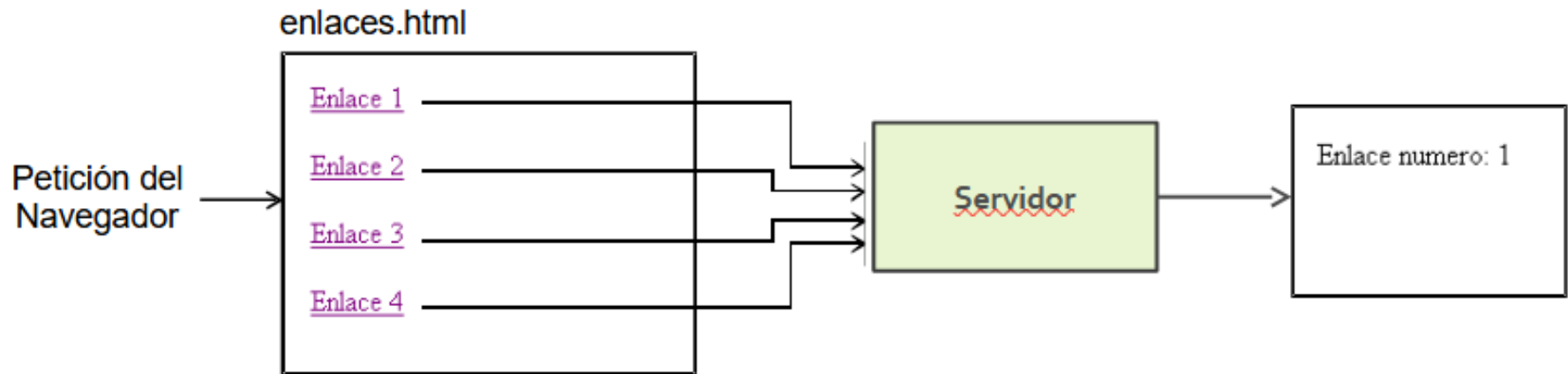
```
http://www.micasa.com/offers?city=Madrid
```

```
router.get('/offers', (req, res) => {
  res.render('offers', {
    city: req.query.city,
  });
});
```

Ejercicio 2

- Crear una página **html** con cuatro enlaces
- Todos los enlaces hacen referencia a un mismo controlador
- En la URL de cada enlace incluimos un parámetro llamado “**linkId**” con valores 1, 2, 3 y 4
- Implementar un **router** que sea llamado al pulsar cualquiera de los enlaces
- Diseñar una **plantilla** que muestre el número del enlace que ha sido pulsado

Ejercicio 2



Proceso de formularios y enlaces

ejem6-link-path

- Insertando información en la URL de los enlaces
 - La información también se pueden incluir como parte de la propia URL, en vez de cómo parámetros

```
http://www.micasa.com/offers/Madrid/
```

```
router.get('/offers/:city', (req, res) => {
  res.render('offers', {
    city: req.params.city,
  });
});
```


Ejercicio 3

- Implementa la misma funcionalidad que el ejercicio 2 pero incluye la información en la propia URL en vez de como parámetros

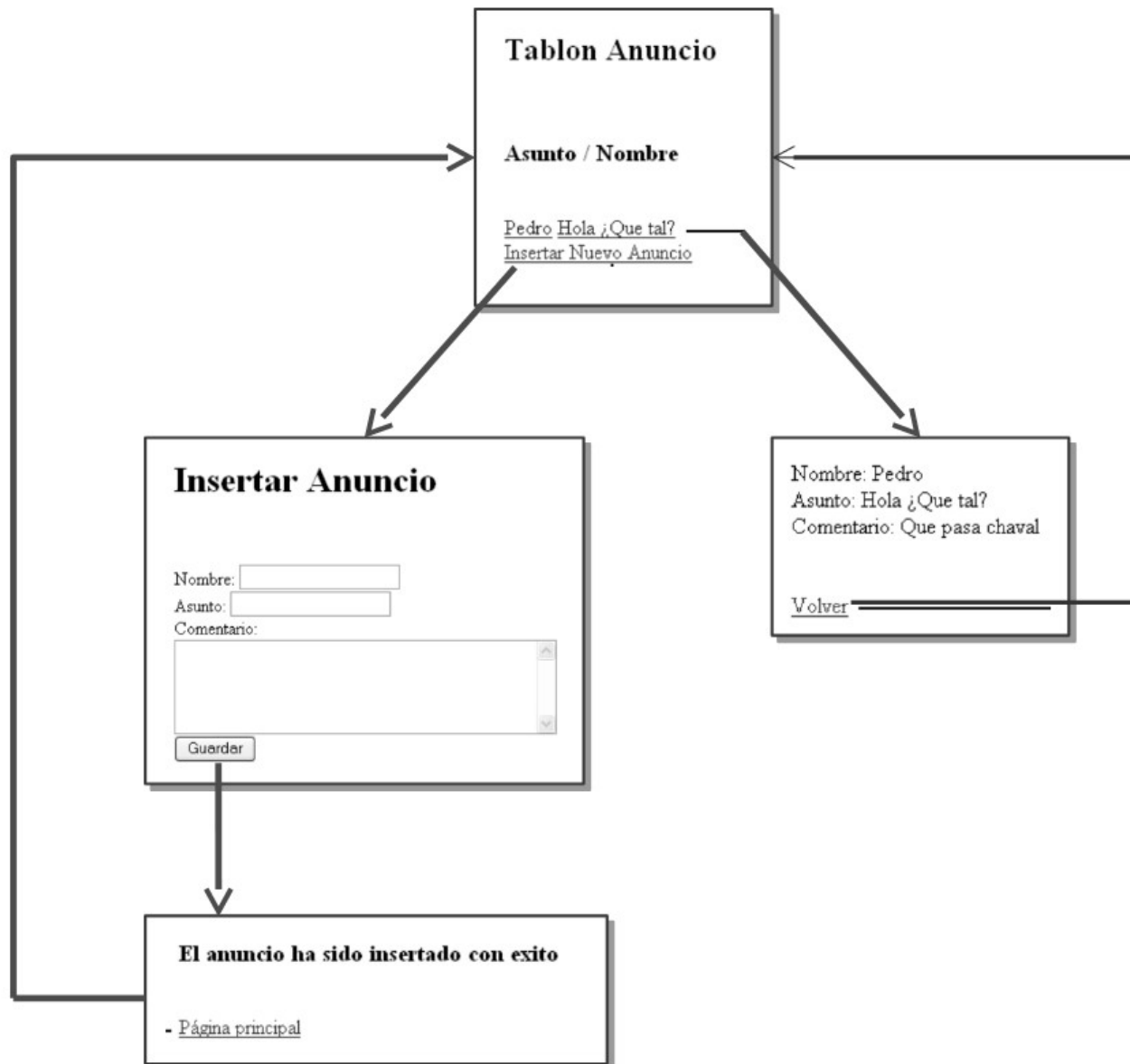
Ejercicio 4 – Tablón de mensajes

- Crear una aplicación web para gestionar un **tablón de anuncios** con varias páginas
- La página principal **muestra los anuncios** existentes (sólo *user* y *title*) y un enlace para insertar un nuevo anuncio
- Si pulsamos en el *title* de un anuncio se navegará a una página nueva que muestre el contenido completo del anuncio

Ejercicio 4 – Tablón de mensajes

- Si se pulsa el enlace para insertar un nuevo anuncio se navegará a una nueva página que contendrá un formulario
- Al enviar el formulario se guardará el nuevo anuncio y se mostrará una página indicando que se ha insertado correctamente, con un enlace para ver ese anuncio y para ir a inicio
- En la página del anuncio habrá un link para borrarlo

Ejercicio 4 – Tablón de mensajes



Ejercicio 4 – Tablón de mensajes

- **Implementación**

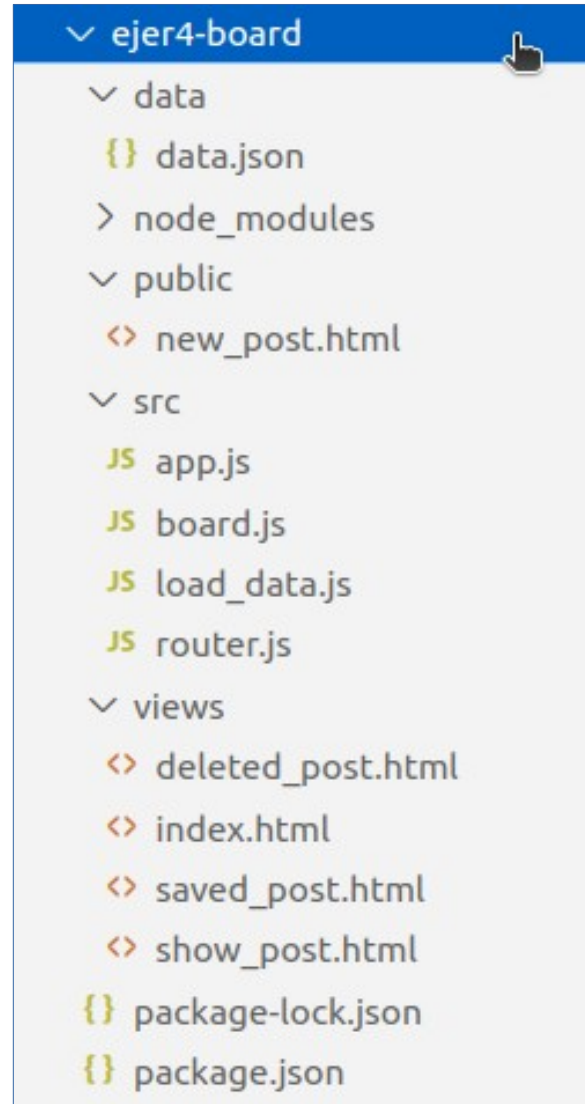
- Se usará un único router con varias funciones (cada una atendiendo una URL diferente)
- Se creará un módulo independiente para la gestión de los anuncios en memoria
- Cada anuncio tendrá asociado un id generado único y se guardará en un mapa

Ejercicio 4 – Tablón de mensajes

- **Implementación**

- Se crearán datos de ejemplo en un fichero `/data/data.json`
- El contenido de ese fichero se cargará en memoria cada vez que se inicie la aplicación

Ejercicio 4 – Solución



Ejercicio 4 – Solución

src/board.js

```
const posts = new Map();
let nextId = 0;

export function addPost(post) {
  let id = nextId++;
  post.id = id.toString();
  posts.set(post.id, post);
}

export function deletePost(id){
  posts.delete(id);
}

export function getPosts(){
  return [...posts.values()];
}

export function getPost(id){
  return posts.get(id);
}
```

- Gestión de información en **memoria**
- Se usa un **mapa** para acceder a los anuncios por su **id**
- El id único se basa en un **contador**

Ejercicio 4 – Solución

src/router.js

```
import express from 'express';
import * as board from './board.js';

const router = express.Router();
export default router;

...

router.get('/', (req, res) => {

  res.render('index', {
    posts: board.getPosts()
  });
});
```

Ejercicio 4 – Solución

views/index.html

```
<html>
<body>
  <h1>Posts</h1>
  {{#posts}}
  <a href="post/{{id}}">{{user}} {{title}}</a><br>
  {{/posts}}

  {{^posts}}
  <p>No posts yet.</p>
  {{/posts}}

  <br>
  <a href="new_post.html">New Post</a>
</body>
</html>
```

Ejercicio 4 – Solución

public/new_post.html

```
<html>
<head>
  <meta charset="UTF-8"/>
</head>
<body>
  <h1>New post</h1>
  <form action="post/new" method="post">
    <p>User: </p>
    <input type='text' name='user' />
    <p>Title:</p>
    <input type='text' name='title' />
    <p>Text:</p>
    <textarea name='text' rows=5 cols=40></textarea>
    <br>
    <input type='submit' value='Save' />
  </form>
</body>
</html>
```

Ejercicio 4 – Solución

src/router.js

```
import express from 'express';
import * as board from './board.js';

const router = express.Router();
export default router;
...

router.post('/post/new', (req, res) => {

  let post = {
    user: req.body.user,
    title: req.body.title,
    text: req.body.text
  };

  board.addPost(post);

  res.render('saved_post', { id: post.id });
});
```

Ejercicio 4 – Solución

views/saved_post.html

```
<html>
<body>
  <p>Post has been saved.</p>
  <a href="/post/{{id}}">View post</a>
  <br>
  <a href="/">Back</a>
</body>
</html>
```

Ejercicio 4 – Solución

views/show_post.html

```
<html>
<body>
  <p>User: {{post.user}}</p>
  <p>Title: {{post.title}}</p>
  <p>Text: {{post.text}}</p>
  <a href="/post/{{post.id}}/delete">Delete post</a>
  <br>
  <a href="/">Home</a>
</body>
</html>
```

Aspectos de seguridad

- Al desarrollar aplicaciones web hay que evitar introducir vulnerabilidades
- El ejercicio 4 es vulnerable a ataques CSRF (*Cross Site Request Forgery*)
- Se debe seguir aprendiendo técnicas de desarrollo web para evitarlos